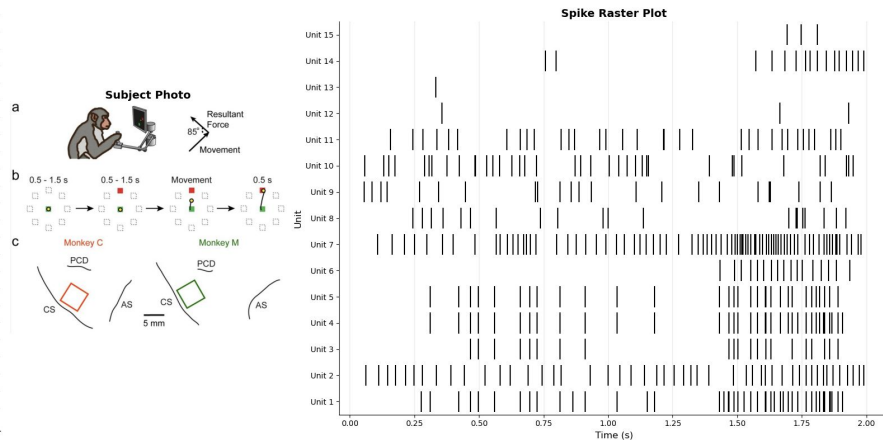
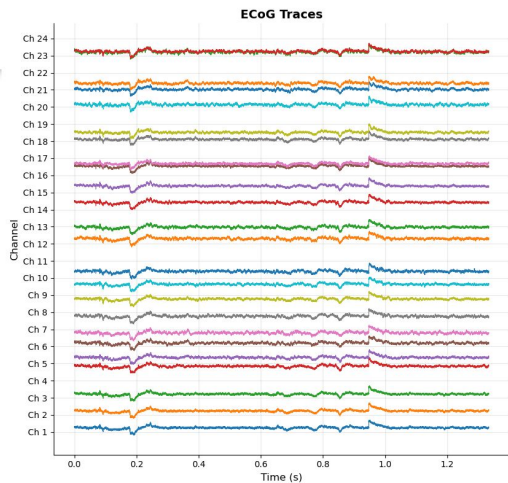
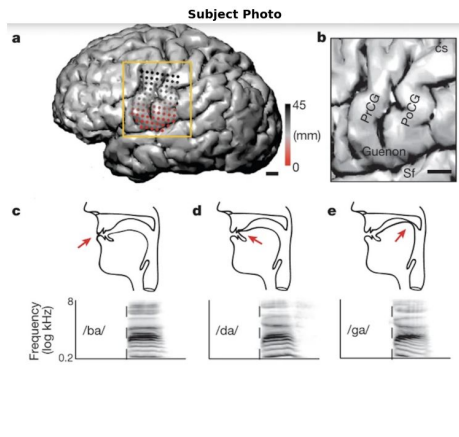




HumanSayMonkeyDo

CSE583
25-12-10

Background + Data Used

Project Goal

Build a reusable analysis pipeline that decodes behavioral events from neural activity and test its generalizability across species

monkey dataset → human dataset

Potential Analysis Directions

- movement onset detection ☒
- tuning curve analysis
- target decoding

Data Used Two NWB datasets from the DANDI Archive

full-scale high-volume!

Monkey dataset

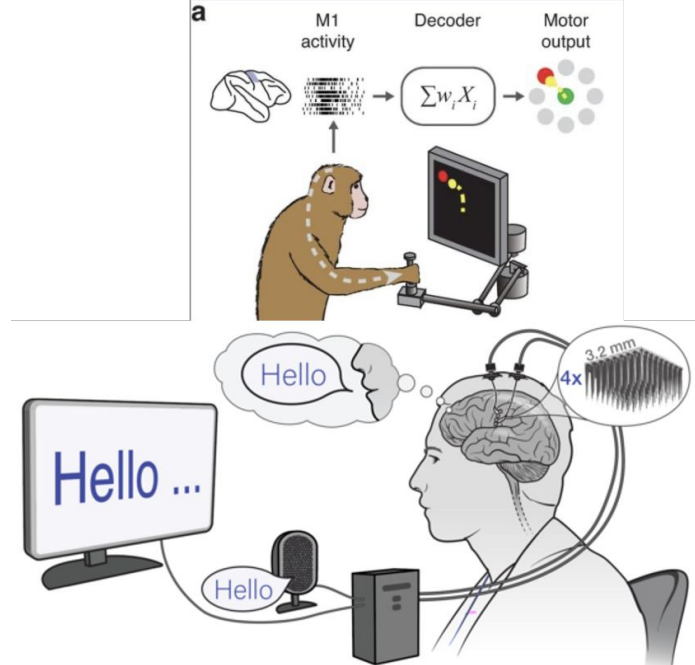
(the primary dataset for building and validating the pipeline)

- center out reaching task
- clear movement onset labels for supervised decoding

Human dataset

(test whether the monkey pipeline generalize across species)

- ECoG (Electrocorticography) speech production task



Use cases

- Find all data files which are available for analysis
- Extract data around important events
- Label the data for classifier training
- Train a movement-onset classifier and evaluate it
- Apply the pipeline to a new NWB session
- Interpreting by visualize neural population activity or classifier results

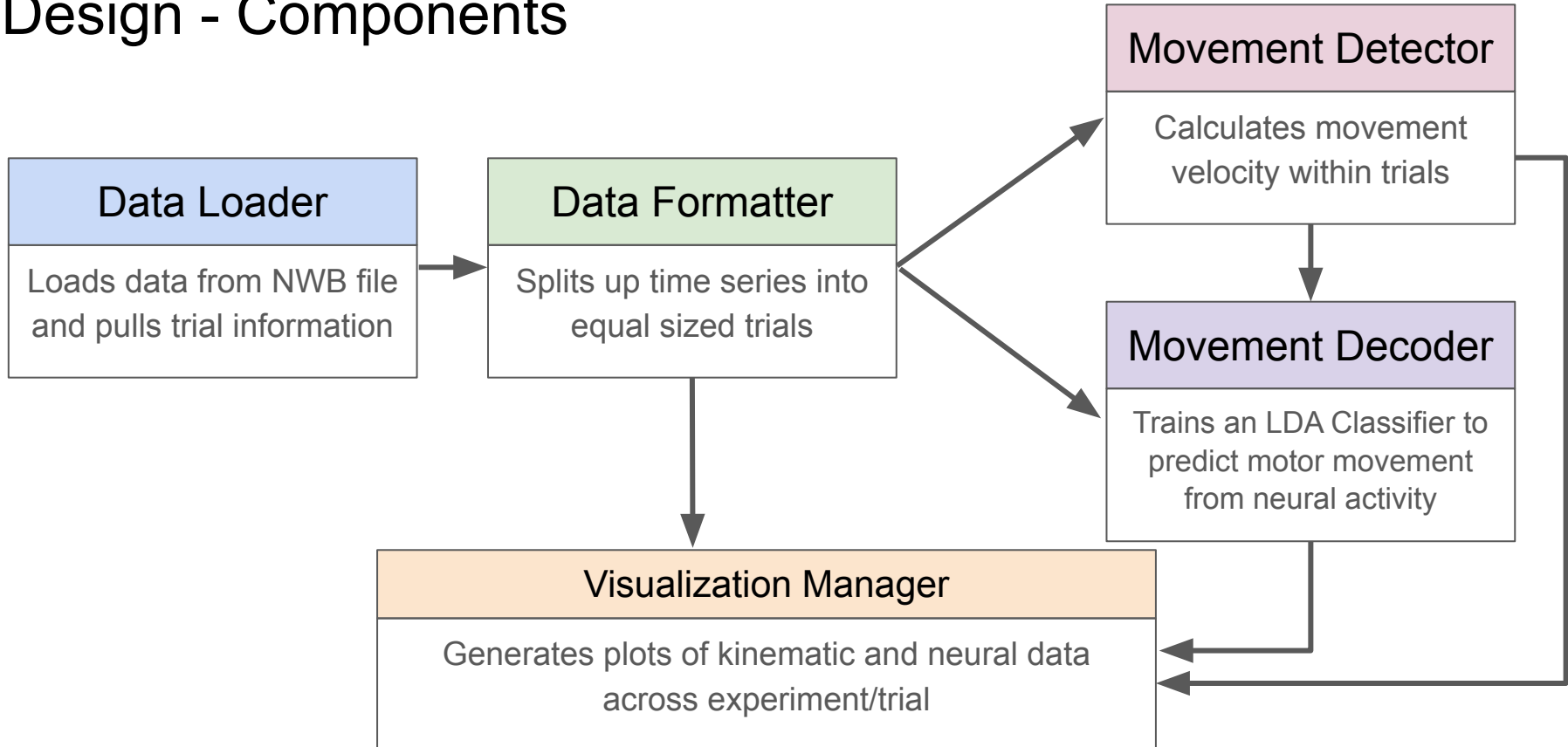


ACTOR



ACTOR

Design - Components



Project structure

Repository Layout

src/	core modules
tests/	test suite
examples/	notebooks
doc/	specs + stories
config.yaml	config (root)
environment.yaml	env (root)

Pipeline Components

Data Loading (NWB I/O)

Data Formatting

Analysis (LDA)

Visualization

Software Practices

Separation of concerns • Reproducible setup • 15+ tests • Installable package

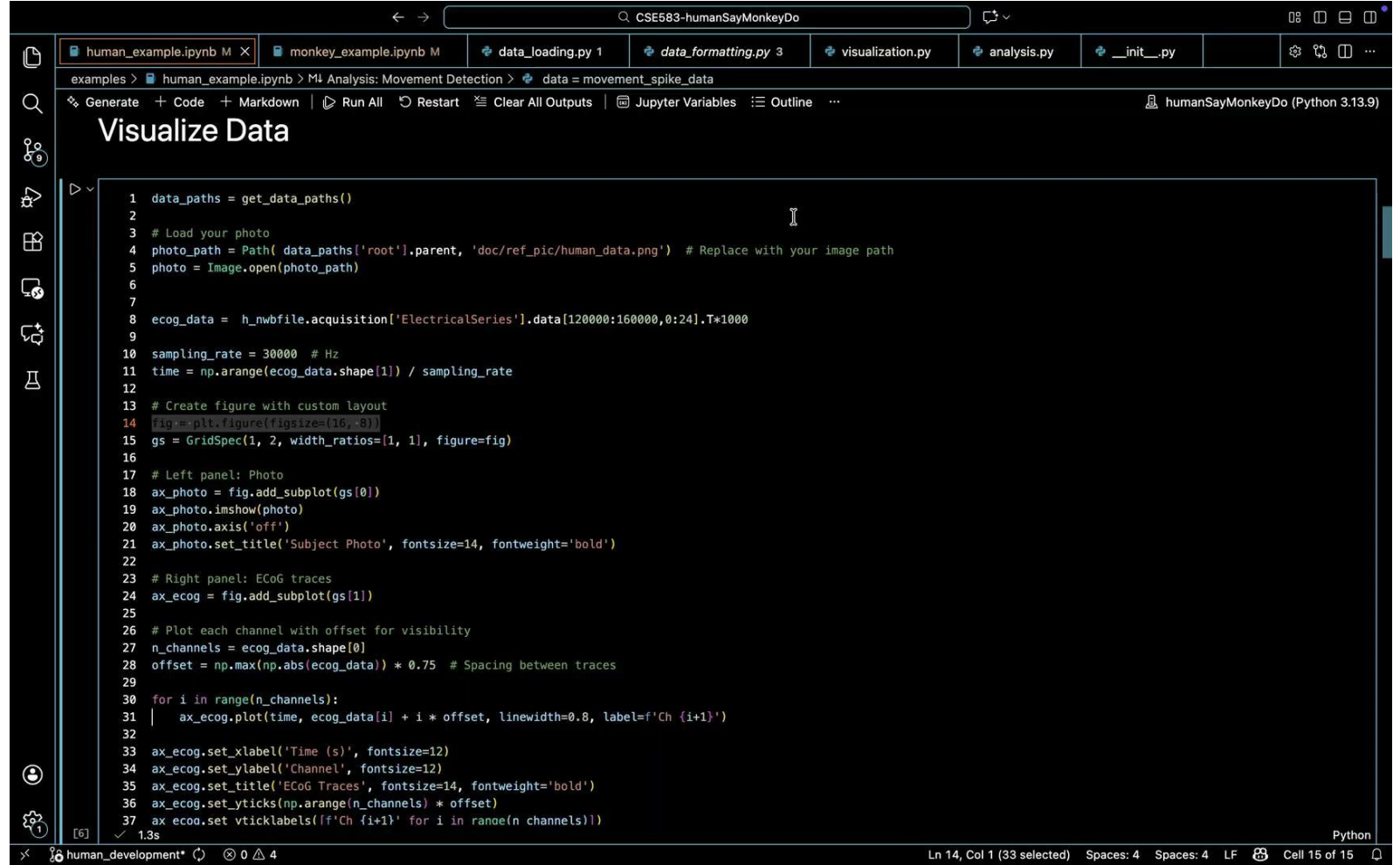
Monkey Data

The screenshot shows a Jupyter Notebook window titled "CSE583-humanSayMonkeyDo". The notebook has two tabs: "human_example.ipynb M" and "monkey_example.ipynb M". The active tab is "monkey_example.ipynb M". The notebook's toolbar includes options like "Generate", "Code", "Markdown", "Run All", "Restart", "Clear All Outputs", "Jupyter Variables", and "Outline". The code cell contains the following Python code:

```
1 import numpy as np
2 from pynwb import read_nwb
3 from matplotlib import pyplot as plt
4 from matplotlib.gridspec import GridSpec
5 from PIL import Image
6
7 from pathlib import Path
8
9 from CSE583_humanSayMonkeyDo.load_config import get_data_paths
10
11 from CSE583_humanSayMonkeyDo.data_loading import(
12     get_nwbbs,
13     get_neural_data,
14     get_trial_times,
15     get_kinematics,
16 )
17
18 from CSE583_humanSayMonkeyDo.visualization import(
19     plot_lda_results,
20     plot_firing_rate_heatmap,
21 )
22
23 from CSE583_humanSayMonkeyDo.data_formatting import(
24     get_windowed_pos_chunk,
25     get_chunk_spikes_binned_windowed,
26 )
27
28 from CSE583_humanSayMonkeyDo.analysis import(
29     get_movement_onset_times,
30     train_lda_classifier,
31 )
```

The code cell is executed, showing a status bar at the bottom left with "[13] ✓ 0.0s" and "Python" at the bottom right. Below the code cell, there is a section titled "Load Monkey Data" with a toolbar containing icons for "Run", "Copy", "Paste", "Clear", and "Delete". The bottom status bar shows "human_development*" and "Spaces: 4 Cell 4 of 13".

Human Data



The screenshot displays a JupyterLab environment with a dark theme. The top bar shows the file browser with open files: `human_example.ipynb`, `monkey_example.ipynb`, `data_loading.py`, `data_formatting.py`, `visualization.py`, `analysis.py`, and `__init__.py`. The active notebook is `human_example.ipynb`, which is in the 'Code' view. The title bar of the notebook indicates the project is 'CSE583-humanSayMonkeyDo' and the Python version is 3.13.9.

The main area of the notebook contains a Python script for visualizing data. The script is as follows:

```
1 data_paths = get_data_paths()
2
3 # Load your photo
4 photo_path = Path(data_paths['root'].parent, 'doc/ref_pic/human_data.png') # Replace with your image path
5 photo = Image.open(photo_path)
6
7
8 ecog_data = h_nwbfile.acquisition['ElectricalSeries'].data[120000:160000,0:24].T*1000
9
10 sampling_rate = 30000 # Hz
11 time = np.arange(ecog_data.shape[1]) / sampling_rate
12
13 # Create figure with custom layout
14 fig = plt.figure(figsize=(16, 8))
15 gs = GridSpec(1, 2, width_ratios=[1, 1], figure=fig)
16
17 # Left panel: Photo
18 ax_photo = fig.add_subplot(gs[0])
19 ax_photo.imshow(photo)
20 ax_photo.axis('off')
21 ax_photo.set_title('Subject Photo', fontsize=14, fontweight='bold')
22
23 # Right panel: ECoG traces
24 ax_ecog = fig.add_subplot(gs[1])
25
26 # Plot each channel with offset for visibility
27 n_channels = ecog_data.shape[0]
28 offset = np.max(np.abs(ecog_data)) * 0.75 # Spacing between traces
29
30 for i in range(n_channels):
31     ax_ecog.plot(time, ecog_data[i] + i * offset, linewidth=0.8, label=f'Ch {i+1}')
32
33 ax_ecog.set_xlabel('Time (s)', fontsize=12)
34 ax_ecog.set_ylabel('Channel', fontsize=12)
35 ax_ecog.set_title('ECoG Traces', fontsize=14, fontweight='bold')
36 ax_ecog.set_yticks(np.arange(n_channels) * offset)
37 ax_ecog.set_vticklabels([f'Ch {i+1}' for i in range(n_channels)])
```

The status bar at the bottom indicates the current position is 'Ln 14, Col 1 (33 selected)' and the file is 'human_development*'. The Python version 'Python 3.13.9' is also shown.

Lessons learned

1. Multiple data types, multiple headaches
2. Limitations of Jupyter notebook development: Moving towards test driven
3. Smaller PR's!!
4. Don't be afraid of merge!

Future Work

1. Additional analysis modules
2. Improved edge case handling